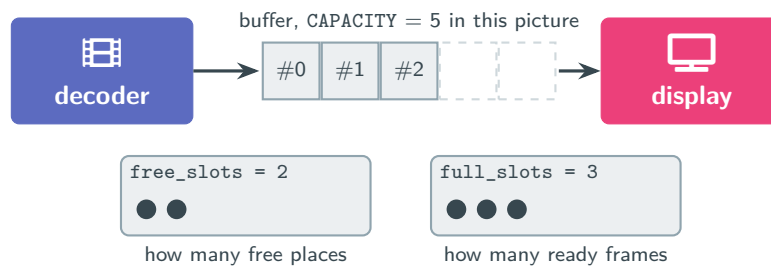


Computer Hardware ■ Lecture 8: Practice with Threads

Task 10: The decoder with two semaphores

Ticket VD-1503, from the e-mail of Tomasz in Task 9: “Try to rewrite the buffer from VD-1482 with two semaphores: one counts the free places, the other one the ready frames. I’d like to compare both versions at the next code review.”

The template is the solution of Task 8, with condition variables. Rewrite it with two semaphores:



- `free_slots`: how many free places are in the buffer;
- `full_slots`: how many decoded frames wait in the buffer.

Keep `std::queue`. A semaphore of this size is declared as `std::counting_semaphore<CAPACITY> name{start value};` (header `<semaphore>`).

The code

The template is in [Task10](#). This is `video.cpp`; `video.h` is the same as in Task 8.

```

const int CAPACITY = 10;           // the buffer holds at most 10 frames

std::queue<Frame> buffer;
bool finished = false;           // the decoder has decoded the last frame
std::mutex m;
std::condition_variable not_full; // the decoder waits here when it is full
std::condition_variable not_empty; // the display waits here when it is empty

void decoder() {                  // the producer
    for (int i = 0; i < FRAMES; i++) {
        Frame f = decode_frame(i);
        std::unique_lock<std::mutex> lock(m);
        while ((int)buffer.size() == CAPACITY)
            not_full.wait(lock); // sleep until the display takes a frame
        buffer.push(f);
        not_empty.notify_one(); // the display may be waiting for a frame
    }
    std::lock_guard<std::mutex> lock(m);
    finished = true;             // nothing more comes
    not_empty.notify_one();      // the display may be asleep
    printf("decoder: all frames decoded\n");
}

void display() {                 // the consumer
    while (true) {
        std::unique_lock<std::mutex> lock(m);
        while (buffer.empty() && !finished)
            not_empty.wait(lock); // until a frame comes or the end
        if (buffer.empty()) {    // finished and nothing left: the end
            return;
        }
        Frame f = buffer.front();
        buffer.pop();
        not_full.notify_one();   // the decoder may be waiting for space
        lock.unlock();
        show(f);                 // 40 ms, not under the mutex
    }
}

int main() {
    std::thread d(decoder), s(display);
    d.join();
    s.join();
    report();
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- ❶ What values do `free_slots` and `full_slots` have at the start? Why?
- ❷ Who calls `acquire()` and who calls `release()` on each of them? Write the new version.
- ❸ With semaphores, do you still need the mutex? What for?
- ❹ A colleague first locks the mutex and only then calls `acquire()` on a semaphore. Write this version and run it. What happens? Why? (Draw who waits for whom.)

- 5 How do you now tell the display that the video is over? Why does the flag `finished` from Task 8 not work so easily here?
- 6 Compare your version with the one with condition variables. Which one is shorter? Which one is easier to read? When would you choose a condition variable, and when a semaphore?